FACULTAD REGIONAL ROSARIO
Universidad Tecnológica Nacional

Sistema de Gestión Hardware Store “IdeWare”

Proyecto Integrador – Tecnologías de Desarrollo de Software IDE

Comisión: 3EK02

Año: 2026

Grupo: T01

Docentes:

Ezequiel Porta

Sebastian Rocco

Integrantes:
50412 - Gil Agustin - agussv360@gmail.com
50475 - Fiorini Mauricio - mauriciofiorini911@gmail.com
51541 - Herrera Nicolás - herreranico2703@gmail.com

Descripción **3**
Requerimientos funcionales (RF) 3
Autenticación y usuarios 3
Catálogo 3
Órdenes de compra 4
Reportes 4
Requerimientos no funcionales (RNF) 5
Seguridad 5
Arquitectura y mantenibilidad 5
Integridad de datos 5
Rendimiento 5
Documentación 6
Capturas de pantalla **7**
Rol Admin 7
Sitio Web 7
Windows Form 8
Rol Usuario 9
Sitio Web 9
Windows Forms 10
Modelos **11**
Modelo de datos **11**
Modelo de clases **12**
Arquitectura Implementada **13**
Tecnologías Utilizadas **14**
Repositorio **14**

Descripción

El sistema consiste en una solución integral de software orientada a la gestión y comercialización de hardware de computadoras y componentes tecnológicos (CPUs, GPUs, RAM, periféricos, etc.). Permite la administración integral del catálogo de productos con sus especificaciones técnicas dinámicas, gestión de marcas, categorías, usuarios con control de roles y el procesamiento de órdenes de compra con detalle de ítems.

La solución está construida sobre **.NET 8** siguiendo una **arquitectura en capas**, donde la lógica de negocio reside en un backend expuesto mediante **ASP.NET Core** con autenticación y autorización basada en tokens **JWT**.
La API es consumida de manera unificada a través de una librería de clientes HTTP compartida (API.Clients) por dos aplicaciones frontend:

1. **Windows Forms**: Aplicación de escritorio para la administración y operación interna.
2. **Blazor Web Assembly**: Aplicación web SPA (Single Page Application) para la navegación, compras y gestión desde el navegador.

Requerimientos funcionales (RF)

Autenticación y usuarios

RF-01: El sistema debe permitir iniciar sesión con mail y contraseña.
RF-02: El sistema debe permitir cerrar sesión. En el escritorio se cierran todas las ventanas abiertas.
RF-03: El sistema debe permitir que un cliente nuevo se registre desde la aplicación de escritorio. Queda siempre con rol Usuario.
RF-04: El sistema debe detectar la sesión vencida y volver a pedir el login.
RF-05: El Admin debe poder dar de alta, modificar, dar de baja y listar usuarios. Al editar, puede cambiar el rol, y la contraseña es opcional (si queda vacía, se mantiene la anterior).
RF-06: El sistema debe permitir buscar usuarios por nombre, apellido o email.
RF-07: El sistema no debe permitir dos usuarios con el mismo email.

Catálogo

RF-08: El Admin debe poder gestionar categorías (alta, baja, modificación y consulta), sin nombres repetidos.
RF-09: El Admin debe poder gestionar marcas (alta, baja, modificación y consulta), sin nombres repetidos.

RF-10: El Admin debe poder gestionar productos: nombre, descripción, precio, stock, pre-venta, categoría y marca. No se permiten nombres ni descripciones repetidos, y la categoría y la marca tienen que estar activas.
RF-11: Los usuarios deben poder ver el catálogo de productos activos. En la web se muestra como tarjetas.
RF-12: El Admin debe poder gestionar las especificaciones técnicas de cada producto (clave, valor y unidad opcional), sin claves repetidas en un mismo producto. Todos los usuarios pueden consultarlas.

Órdenes de compra

RF-13: Un usuario debe poder crear una orden con uno o más productos, sus cantidades y una dirección de envío. El total se calcula con el precio vigente al momento de la compra.
RF-14: Al confirmar la orden, el sistema debe validar el stock (salvo en productos en pre-venta) y descontarlo.
RF-15: Un usuario debe poder consultar el historial de sus propias órdenes.
RF-16: El Admin debe poder: ver todas las órdenes, crear órdenes para cualquier usuario, modificar el estado y la dirección de una orden (no los ítems ni el total), dar de baja órdenes.
RF-17: Las órdenes deben pasar por los estados Pendiente, Procesando, Enviado, Entregado y Cancelado.

Reportes

RF-18: El Admin debe poder generar un informe de ventas filtrando por rango de fechas, estado de la orden y cantidad de productos del top.
RF-19: El informe debe mostrar la cantidad de órdenes, el total facturado y el ticket promedio.
RF-20: El informe debe mostrar las ventas por mes, los productos más vendidos y la facturación por marca con su porcentaje.
RF-21: El informe debe incluir gráficos de barras, tanto en pantalla (web y escritorio) como en el PDF.
RF-22: El sistema debe permitir exportar el informe a PDF.
RF-23: El informe debe excluir siempre las órdenes eliminadas, y también las canceladas cuando no se filtra por estado.
General
RF-24: Las bajas de todas las entidades deben ser lógicas: los registros se marcan como inactivos y no se borran.

Requerimientos no funcionales (RNF)

Seguridad

RNF-01: La autenticación debe usar tokens JWT firmados con HMAC-SHA256, con vencimiento configurable (hoy, 30 minutos).
RNF-02: Las contraseñas deben guardarse con hash PBKDF2 (SHA-256, 10.000 iteraciones) y un salt aleatorio de 32 bytes por usuario. La API nunca debe devolver la contraseña.
RNF-03: Cada endpoint debe exigir un permiso específico (por ejemplo productos.leer u órdenes.agregar), asignado según el rol del usuario.
RNF-04: Todas las consultas a la base deben estar parametrizadas (EF Core, y ADO.NET con SqlParameter tipados), para prevenir SQL injection.
RNF-05: Ante un login fallido, el sistema debe responder lo mismo si el email no existe o si la contraseña es incorrecta, para no revelar qué usuarios existen.

Arquitectura y mantenibilidad

RNF-06: El sistema debe tener una arquitectura en capas (dominio, datos, servicios, API y presentación), separada en 9 proyectos.
RNF-07: La lógica de negocio debe vivir solo en la API, compartida por los dos frontends (escritorio y web) a través de API.Clients.
RNF-08: Las reglas de validación deben estar en las entidades del dominio (setters privados con validaciones).
RNF-09: Se deben usar DTOs para no exponer las entidades del dominio, e inyección de dependencias con interfaces para repositorios y servicios.

Integridad de datos

RNF-10: Las relaciones entre tablas deben impedir borrar registros referenciados (DeleteBehavior.Restrict). Esto, junto con las bajas lógicas, preserva el historial de órdenes.
RNF-11: El alta de una orden debe ser atómica: el descuento de stock y la orden se graban en una única transacción.

Rendimiento

RNF-12: Los reportes deben resolverse con consultas SQL de agregación en ADO.NET, de modo que los cálculos se hagan en la base de datos y no en memoria.
Plataforma y configuración
RNF-13: El sistema debe funcionar con .NET 8 y SQL Server o LocalDB. El cliente de escritorio requiere Windows, y la web un navegador moderno.

RNF-14: La cadena de conexión y los parámetros del JWT deben configurarse en el appsettings.json de la API, y la URL de la API en el appsettings.json de cada cliente.
RNF-15: La base de datos debe crearse automáticamente al iniciar la API, con datos de prueba.
Usabilidad
RNF-16: La interfaz debe estar en español, con formato de moneda y fecha argentina ($, dd/MM/yyyy).
RNF-17: Los menús y botones deben mostrarse según el rol del usuario, y los formularios deben validar los datos antes de enviarlos, con mensajes de error claros.

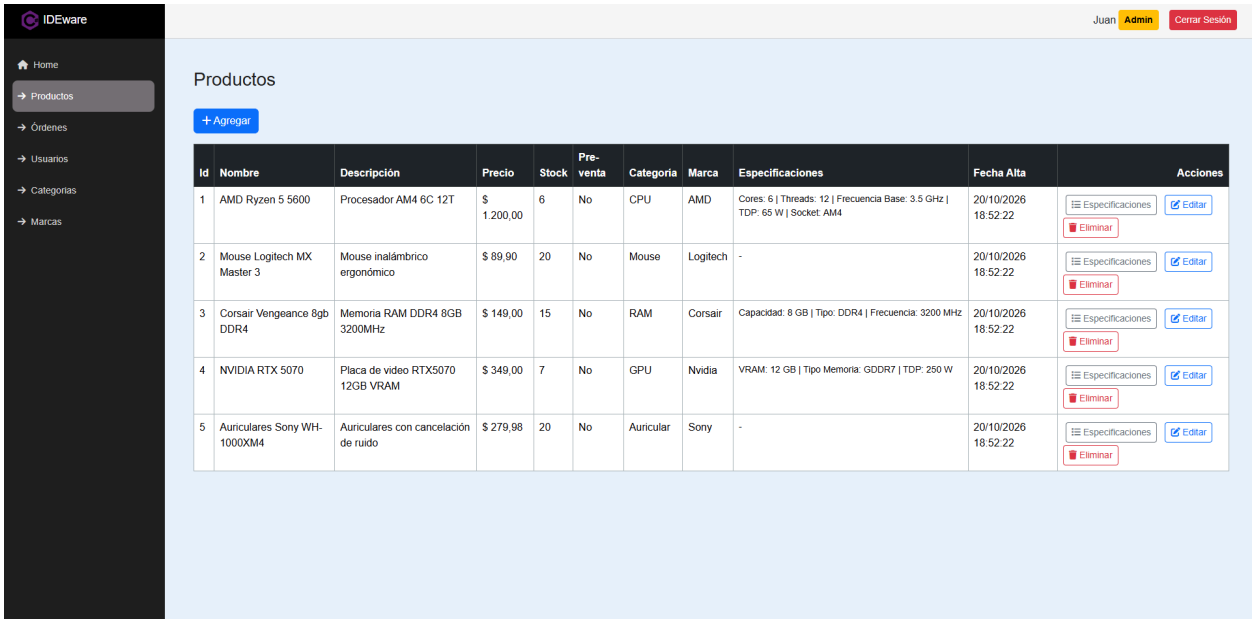
Documentación

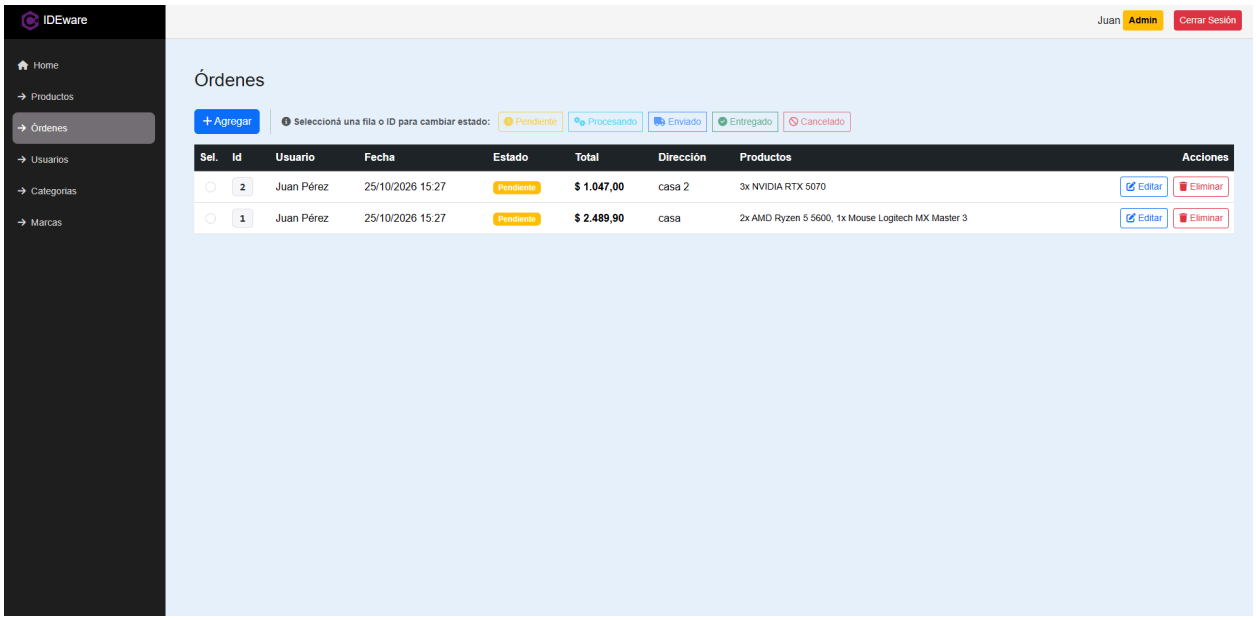
RNF-18: La API debe estar documentada con Swagger/OpenAPI.

Capturas de pantalla

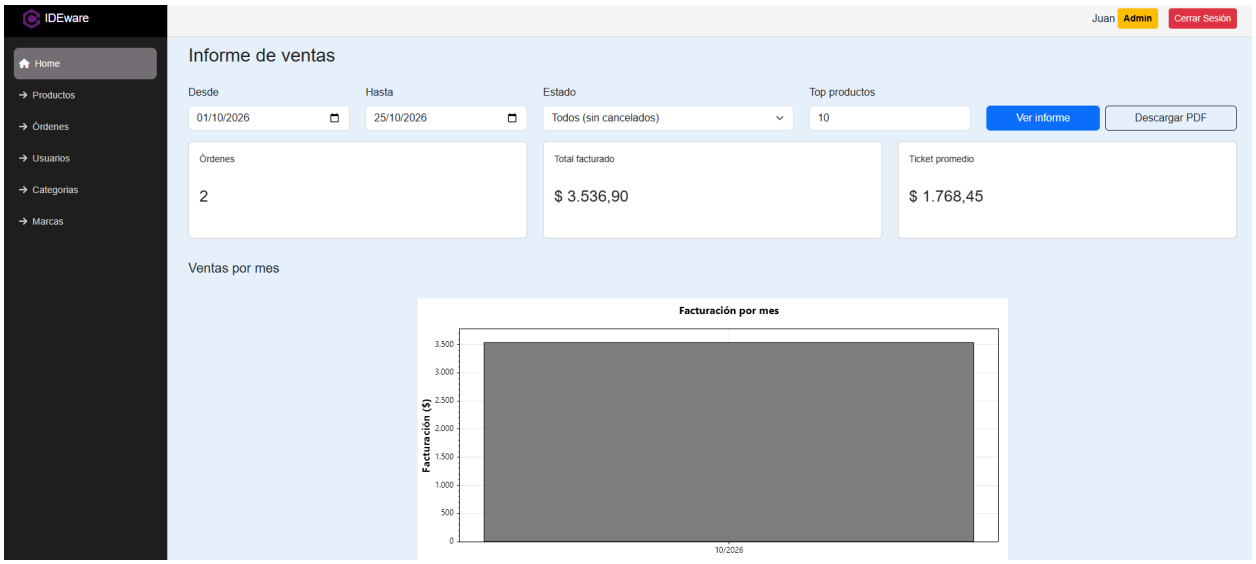
Rol Admin

Sitio Web

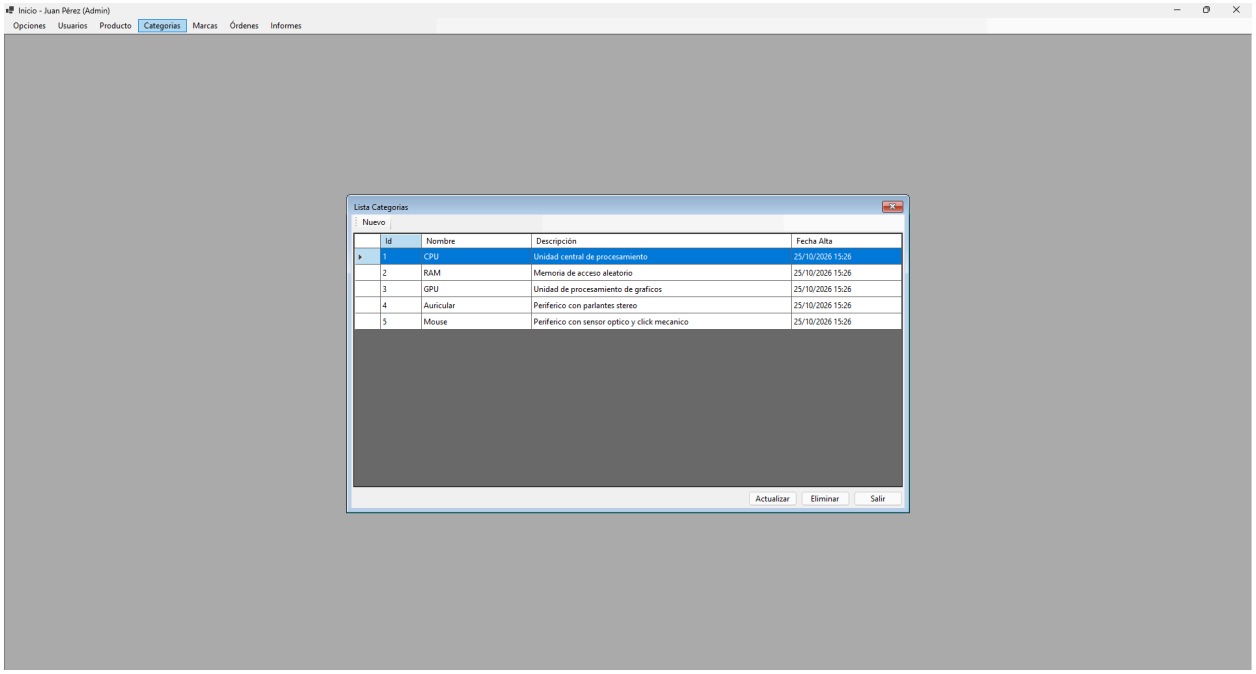




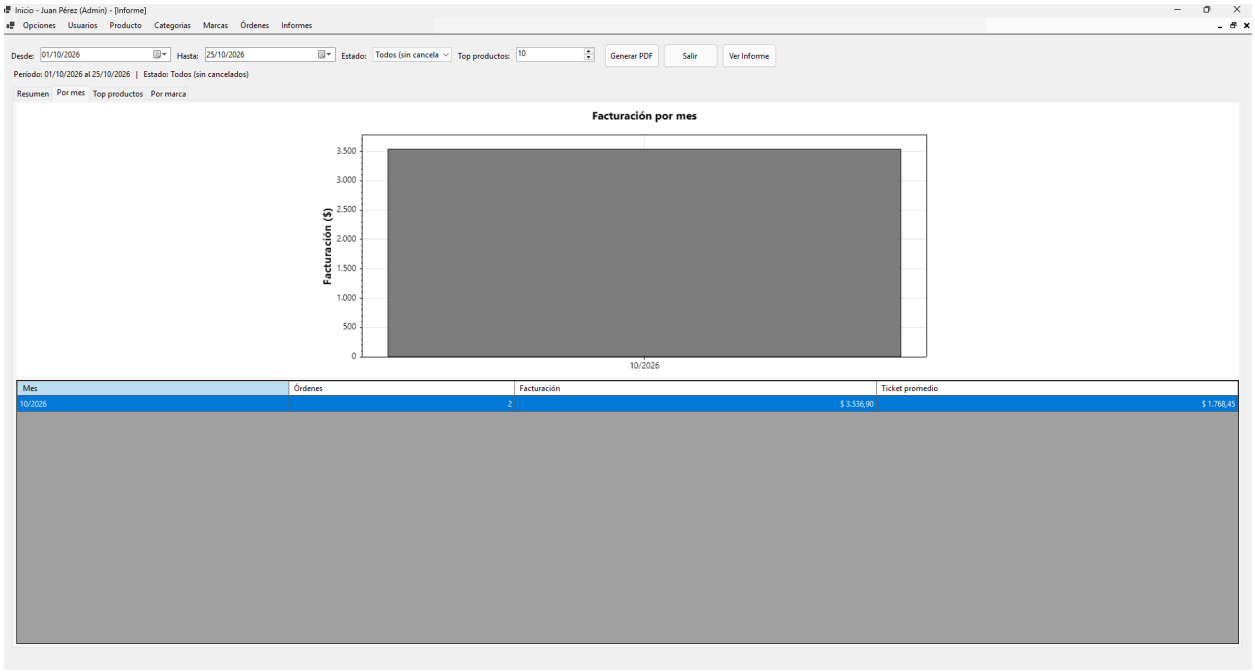
Tecnologías de Desarrollo de Software IDE



Windows Form

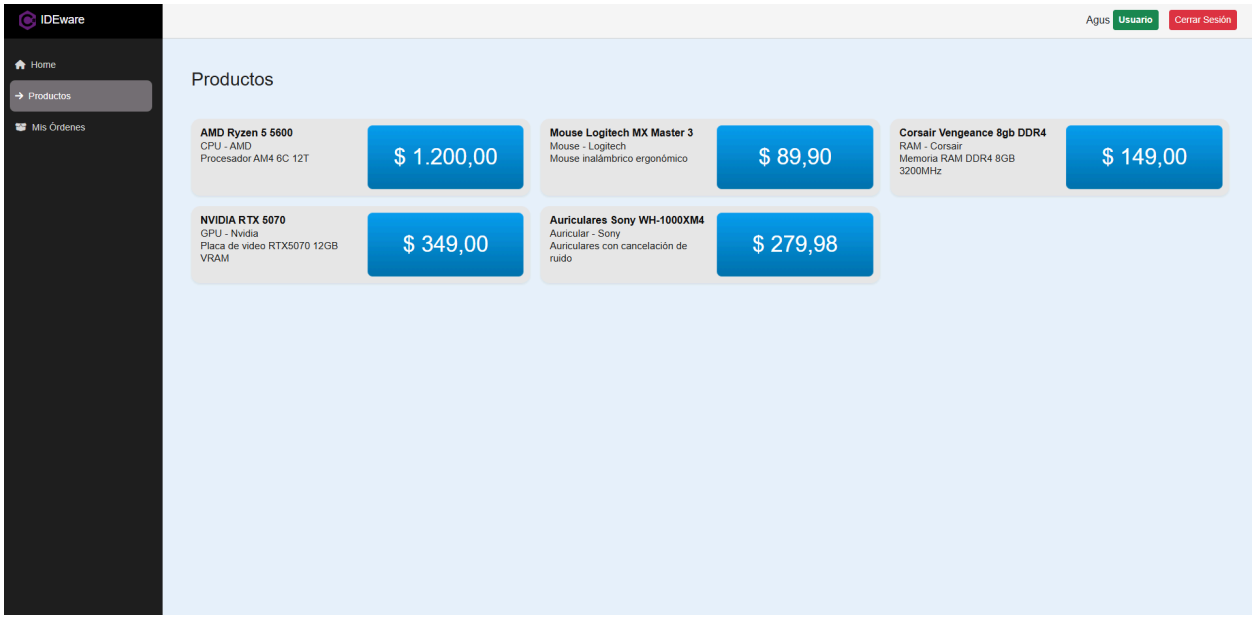


Tecnologías de Desarrollo de Software IDE

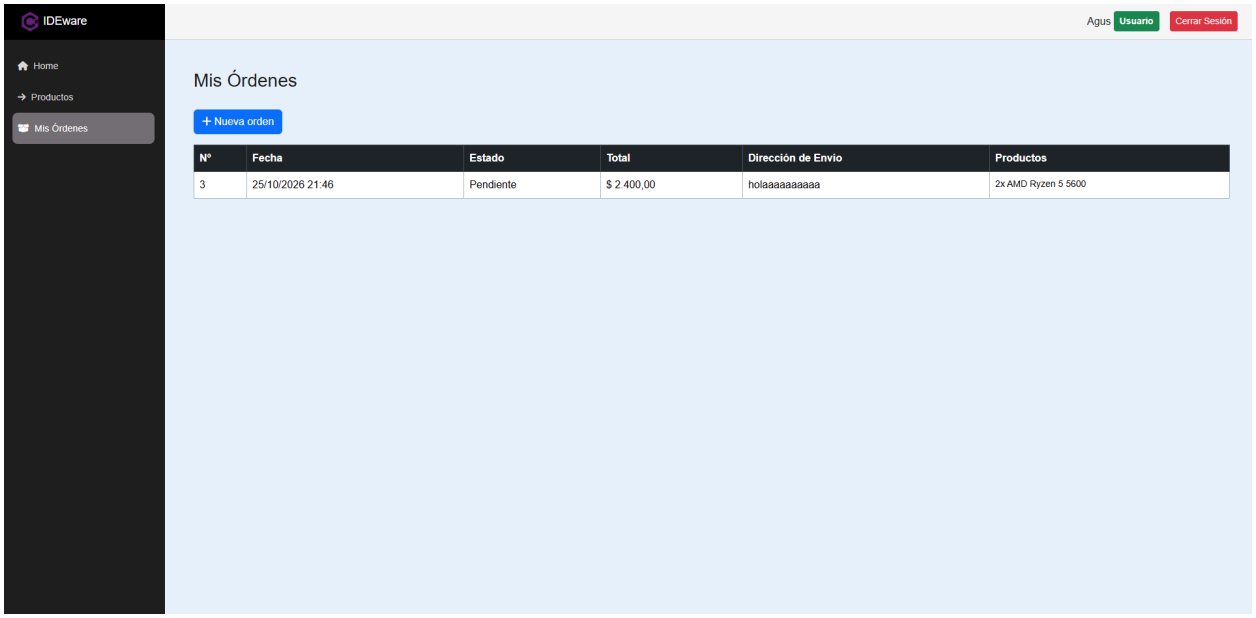


Rol Usuario

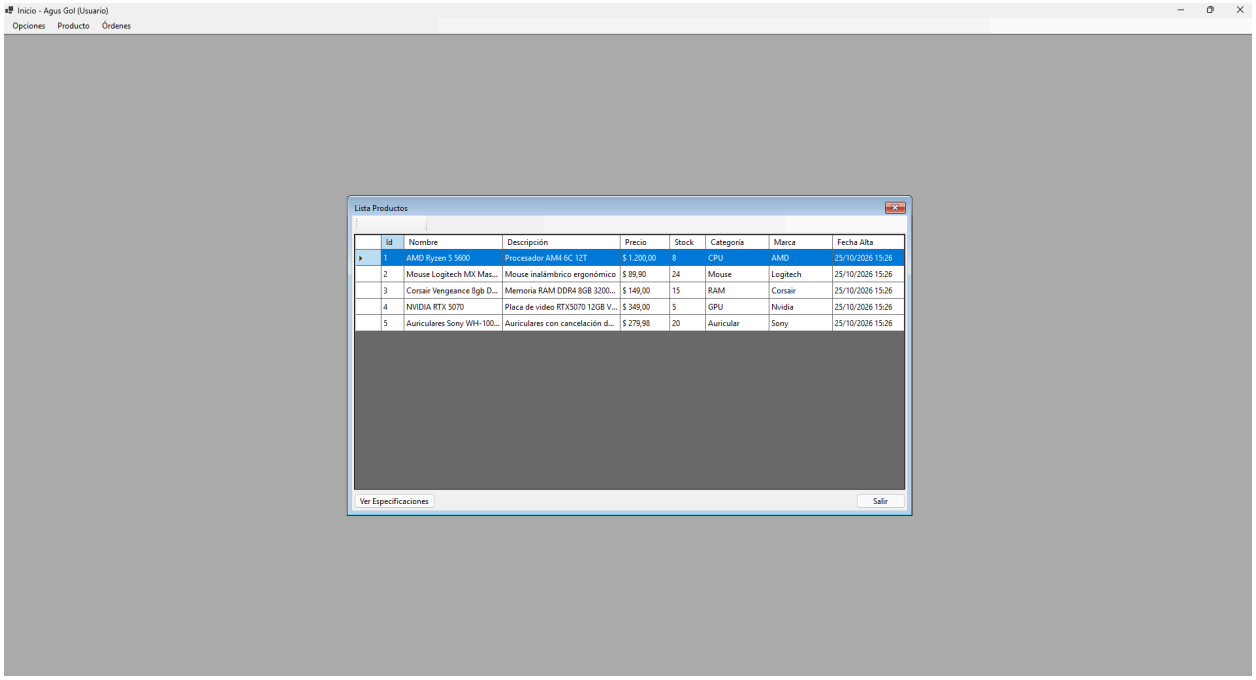
Sitio Web



Tecnologías de Desarrollo de Software IDE

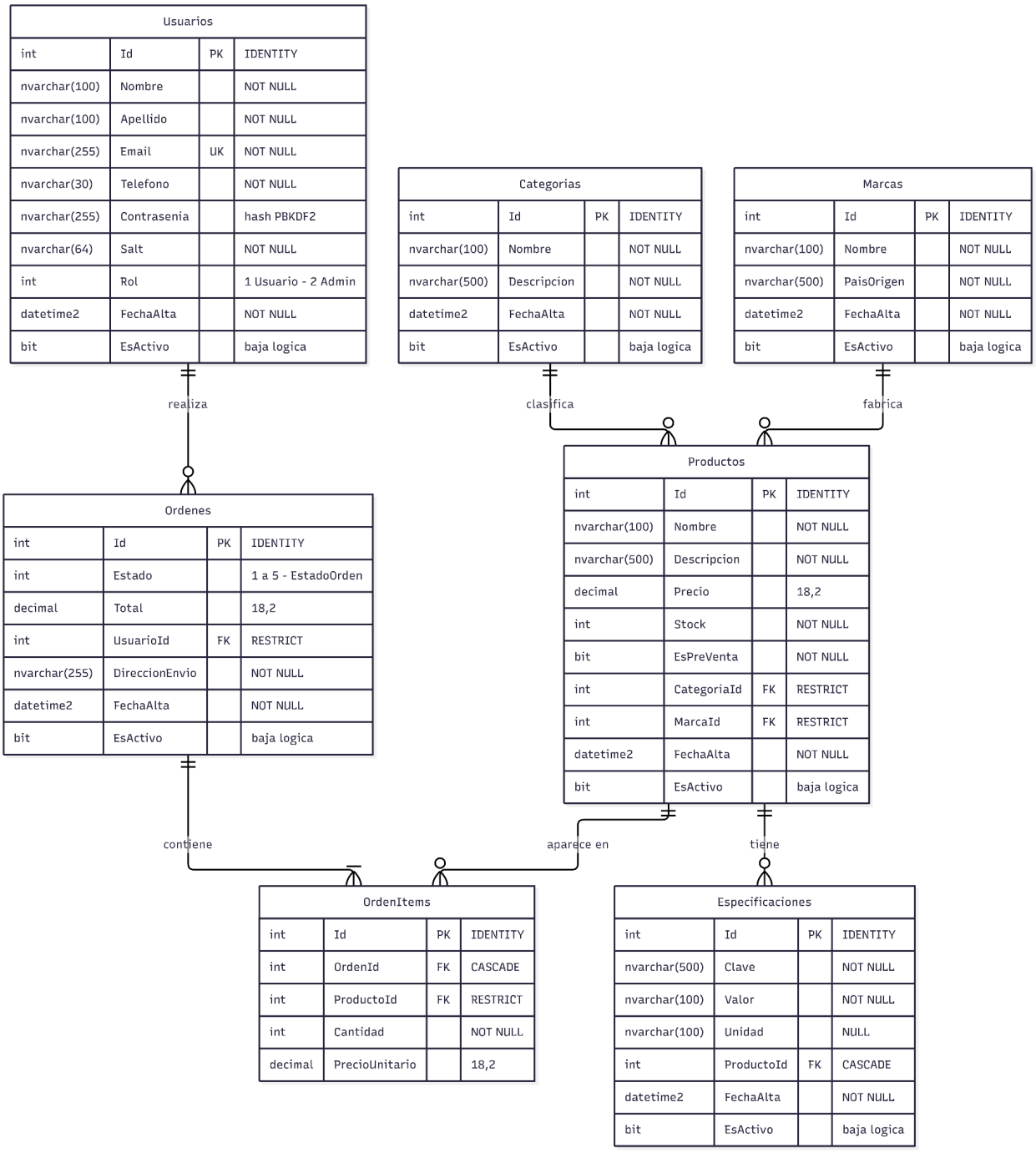


Windows Forms

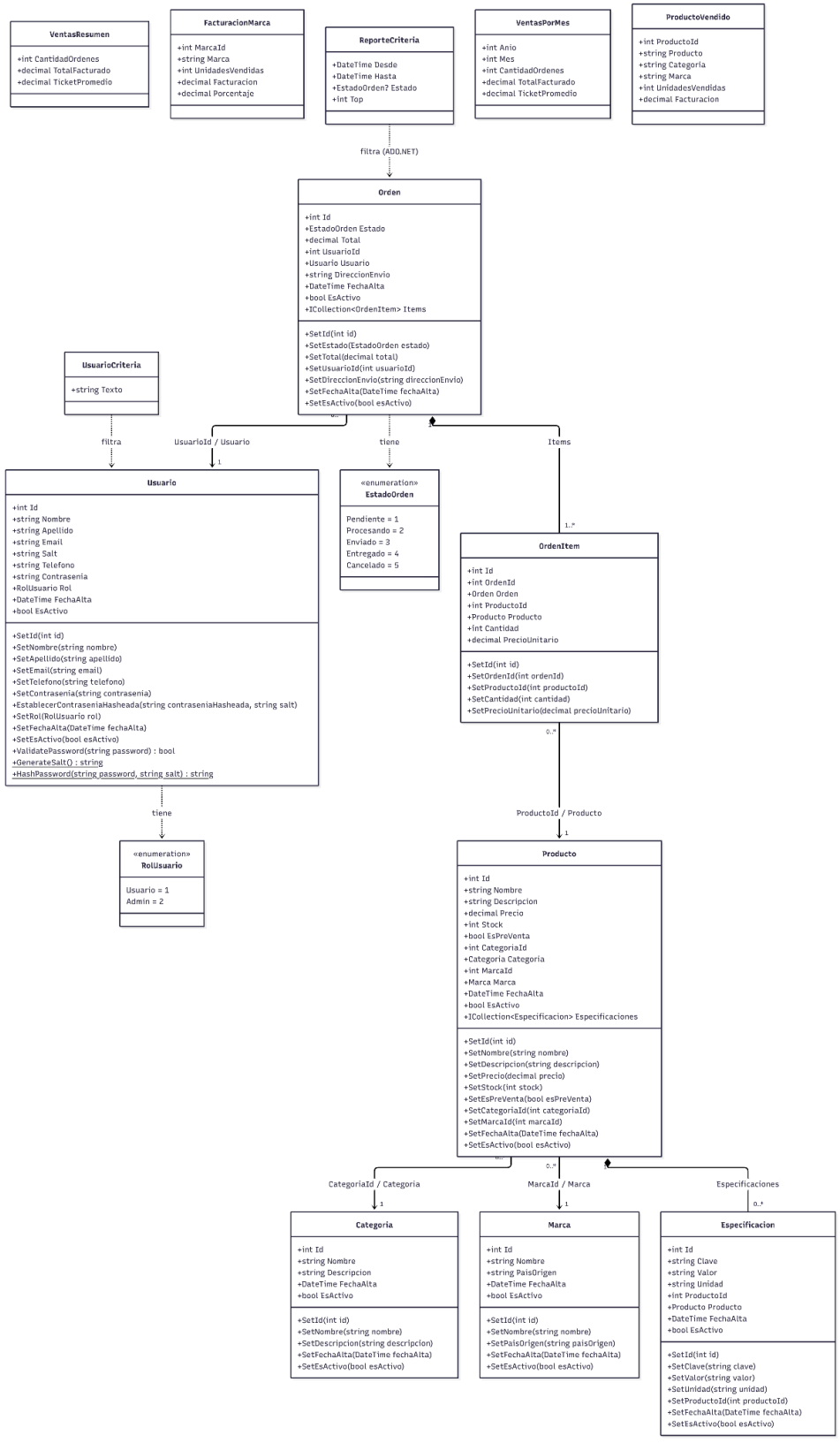


Modelos

Modelo de datos

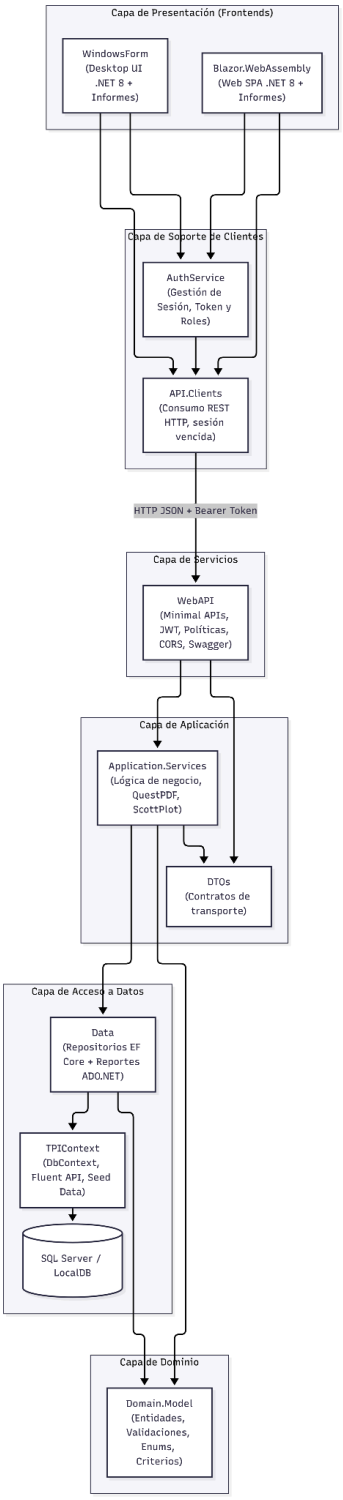


Modelo de clases



Arquitectura Implementada

La arquitectura de N-capas (o N-Tier) es un estilo de diseño de software que divide una aplicación en zonas lógicas (y a veces físicas) independientes, donde cada capa tiene una responsabilidad única y se comunica solo con sus adyacentes



1. **Domain.Model**: Entidades del dominio con encapsulamiento (setters privados), métodos con validaciones, lógica de hash y verificación de contraseñas, modelos de criterios y de resultados de reportes.
2. **DTOs:** Clases utilizadas para la comunicación entre capas y la serialización JSON en la API, evitando exponer directamente las entidades del dominio.
3. **Data:** *persistencia híbrida*, EF Core (TPIContext) para operaciones transaccionales mediante el patrón Repository, y ADO.NET (ReporteRepository) para las consultas de los reportes.
4. **Application.Services:** Servicios que coordinan repositorios y DTOs, validan reglas de negocio y generan los reportes.
5. **WebAPI:** Aplicación ASP.NET Core que expone los endpoints HTTP organizados por entidad y de reportes, autentica mediante JWT y protege cada recurso según los permisos del usuario.
6. **API.Clients:** Librería compartida de clientes HTTP que encapsula las llamadas a la WebAPI, envía el token en la cabecera Authorization: Bearer <token>, toma la URL de la API desde la configuración de cada aplicación y detecta la sesión vencida .
7. **AuthService:** Manejo centralizado del estado de la sesión en el cliente (token, datos del usuario autenticado, eventos de login/logout y comprobación de rol de administrador).
8. **Windows Forms:** Aplicación de escritorio con login, auto-registro, listado, formularios de alta/edición e informe de ventas con pestañas de resumen, gráficos y tablas, y exportación a PDF.
9. **Blazor.Web Assembly**: Aplicación web basada en componentes Razor para administración, catálogo de clientes, compras (MisOrdenes) y el informe de ventas (/informes) con gráficos y descarga de PDF.

Tecnologías Utilizadas

Lenguaje: C# 12
Plataforma / Runtime: .NET 8.0
Backend: ASP.NET Core Minimal APIs
ORM y persistencia transaccional: Entity Framework Core 9.0 (SQL Server Provider, Fluent API, EnsureCreated)
Acceso a datos para reportes: ADO.NET (Microsoft.Data.SqlClient)
Generación de PDF: Quest PDF (licencia Community)
Gráficos: Scott Plot 5 (imágenes PNG generadas en el servidor)
Base de Datos: Microsoft SQL Server / LocalDB
Seguridad: JWT Bearer (Microsoft.AspNetCore.Authentication.JwtBearer), hashing PBKDF2 SHA256 con Salt
Documentación de API: Swagger / OpenAPI (Swashbuckle)
FE-Escritorio: Windows Forms (.NET 8)
FE-Web: Blazor Web Assembly (.NET 8 SPA)
Comunicación: HttpClient compartido con System.Net.Http.Json

Repositorio

https://github.com/starboyagus/IDE-TPI

En este repositorio se pueden ver más a detalle los gráficos de Modelos de datos, tablas y la arquitectura implementada, además teniendo guia de instalación y los usuarios predeterminados de prueba con rol Usuario y Admin.

Código fuente adjunto comprimido